EECE 2311

Lab for EECE2310: Introduction to Digital Design and Computer Architecture

SPRING 2026

Lab Assignment #05

Digital Counters Design

John Kimani

EECE2311 Lab Section #

Sijing Deng

Date Submitted:

Mar 17 2026

Abstract

This project initiates with a fundamental hardware interface, mapping the state of physical pushbuttons to LED outputs to establish basic input/output (I/O) connectivity.

Introduction

This design is progressively scaled to develop a sophisticated digital counter system. The final implementation utilizes integrated switches and pushbuttons to manage complex control logic, including **start/stop** functionality, **asynchronous loading** of counter values, and **bidirectional** (up/down) counting capabilities. Through this incremental approach, the lab demonstrates the practical application of sequential logic and control signaling in an FPGA environment.

Lab Setup

Pre-Lab

As shown in Figure 1, a clock signal is a periodic signal, constantly transitioning between logic values 0 and 1 with symmetric pulses, as shown in the timing diagram. The following properties of a clock signal can be highlighted:

- **A rising edge is a transition from a value of 0 to a value of 1.**
- **A falling edge is a transition from a value of 1 to a value of 0.**
- **The clock period is defined as the time between one rising edge to the next (or one falling edge to the next). The clock period of the DE1-SoC board is 0.02µs or 20ns or 2 x 10^{-8} second.**
- **The clock frequency is defined as the reciprocal of the clock period. Therefore, the clock frequency of the DE1-SoC's FPGA is 50MHz.**

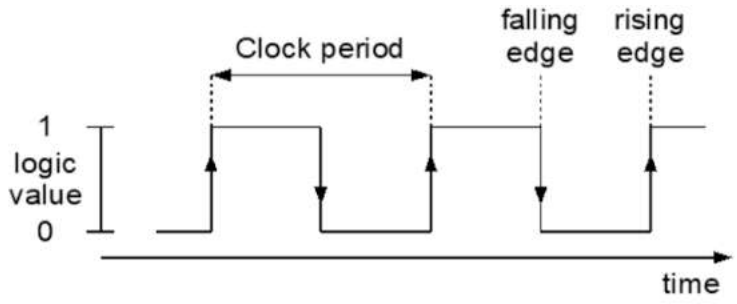


Figure 1: Clock signal

Clock signals are typically connected to all storage components (i.e., the flip flops) of the sequential circuits in a digital design. Their purpose is synchronizing the transition of these

components into their next state. This transition happens at either the rising or falling (triggering) edge of each clock cycle.

An **n-bit counter** is a logic component storing an n-bit binary number that is incremented (up), or decremented (down), on every triggering edge of a clock signal.

An **up free-running (plain binary) counter** is a counter that takes all possible n-bit values incrementally (from 0 to 2^n-1) and is reset to 0 when it reaches its max value.

Its counterpart is a **limited-range (Modulus) counter**, which resets at a configured value (not the default the max value, which is 2^n-1).

The Quartus LPM_COUNTER IP core is a binary counter that can be used to generate up and down counters with outputs of up to 256 bits wide.

Figure 2 shows the ports (interface points) for the LPM_COUNTER IP core. The counter can be configured following the same steps used to configure the other IP Catalog library components we used in the previous lab.

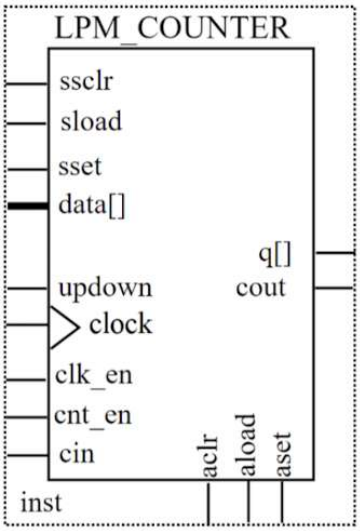


Figure 2: LPM_COUNTER Ports

Equipment

DE1-SoC Development Board
7-Segment Displays
50MHz Clock
Quartus Prime Lite
LPM_COUNTER IP Core

ON/OFF Toggling an LED with a Pushbutton

Using a simple 1-bit counter whose output is connected directly to the LED to configure the DE1-SoC to control the state of an LED using the KEY0 pushbutton: connect a pushbutton directly to an LED, the LED will **turn ON** as long as the **button is not pushed**, and it will **turn OFF** as long as the **button is pushed**.

A 1-bit counter is one that can take only values 0 and 1. When it reaches its maximum value 1, the counter overflows and goes back to 0. Connect the clock of this counter to the pushbutton through an inverter (as the button is active low) as shown in Figure 3. This way, pushing the button will trigger the counter with the positive edge on its clock making it count up by one

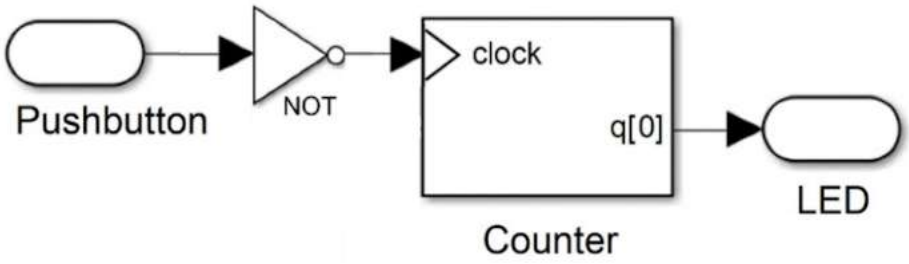


Figure 3: A counter for LED toggle

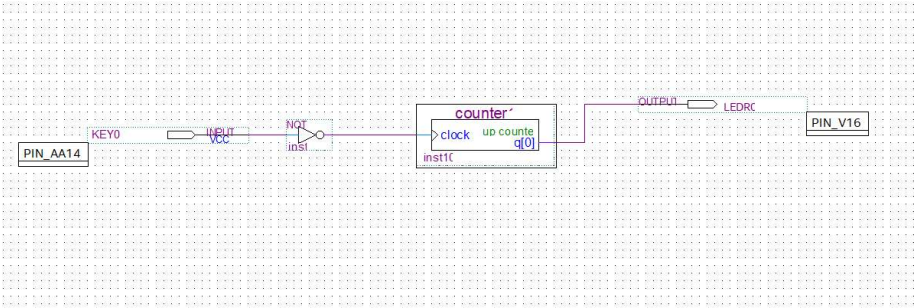


Figure 4: Schematic for LED toggle counter

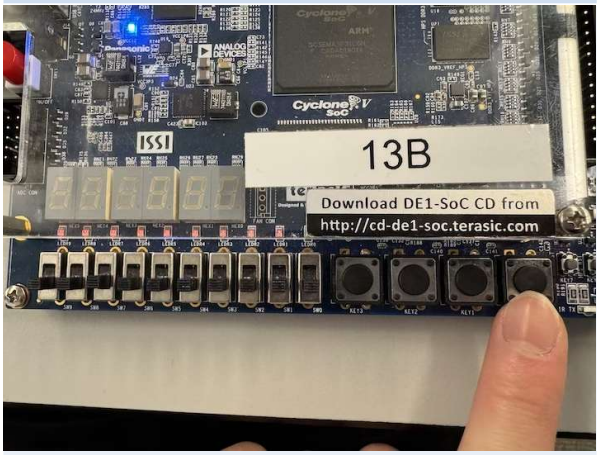
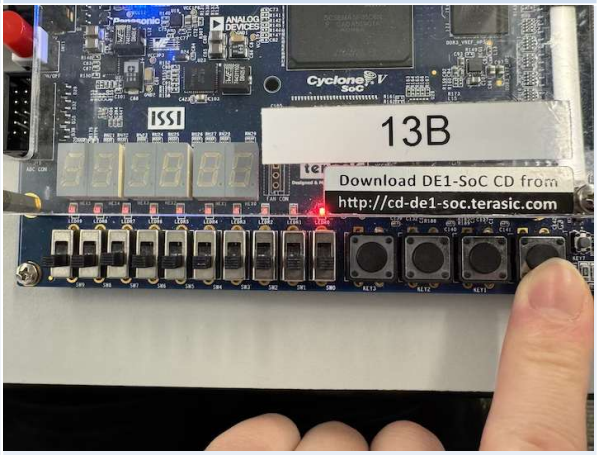
TOGGLE OFF STATE	TOGGLE ON STATE
press KEY[0] k times	press KEY[0] k + 1 times
	

Table 1: LED toggle counter result

Pushbutton-Controlled 4-bit Counter

Design a pushbutton-controlled 4-bit counter. The output of this counter will be displayed on four LEDs. Here the design is similar to the one shown in Figure 3 but with a 4-bit counter instead of a 1-bit counter.

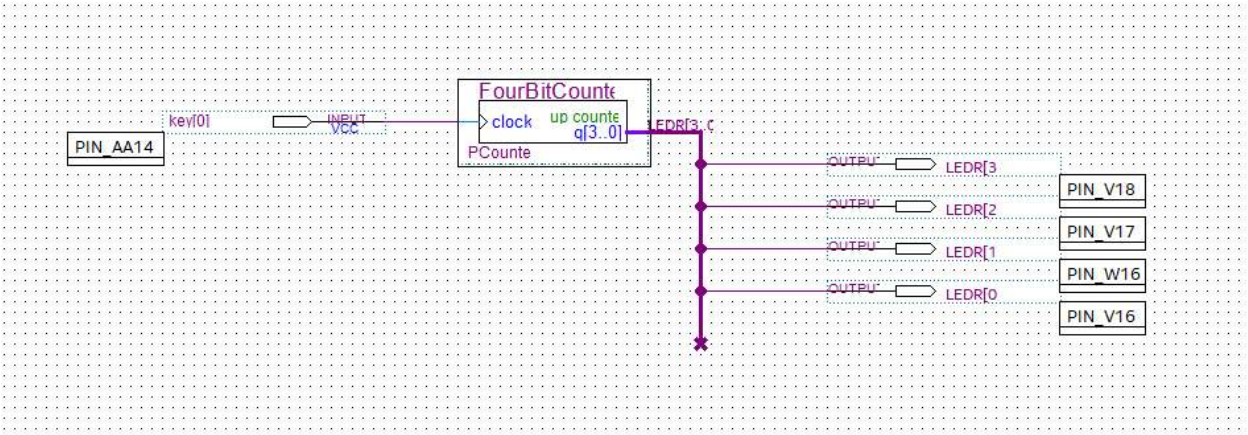


Figure 5: Schematic for **Pushbutton-Controlled 4-bit Counter**

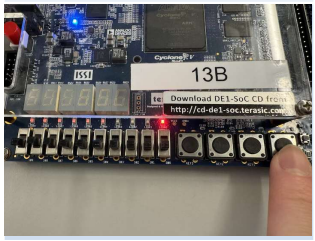
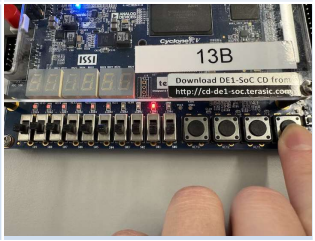
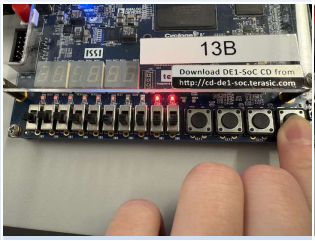
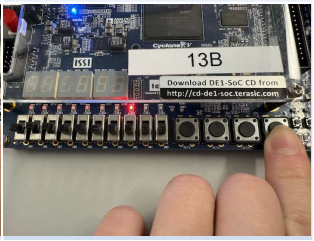
pushbutton #1	pushbutton #2	pushbutton #3	pushbutton #4
press KEY[0] 1+16k time	press KEY[0] 2+16k time	press KEY[0] 3+16k time	press KEY[0] 4+16k time
0001	0010	0011	0100
			

Table 2: **Pushbutton-Controlled 4-bit Counter Result**

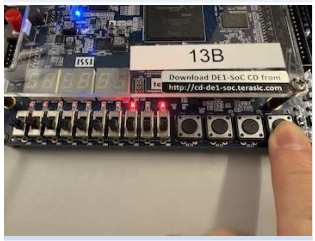
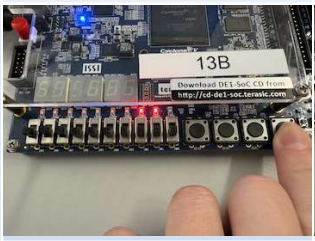
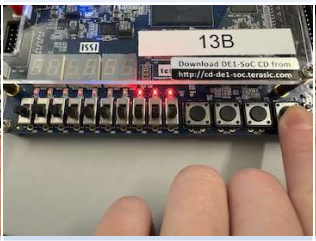
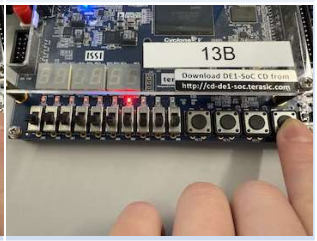
pushbutton #5	pushbutton #6	pushbutton #7	pushbutton #8
press KEY[0] 5+16k time	press KEY[0] 6+16k time	press KEY[0] 7+16k time	press KEY[0] 8+16k time
0101	0110	0111	1000
			

Table 3: **Pushbutton-Controlled 4-bit Counter Result**

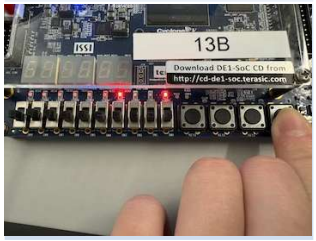
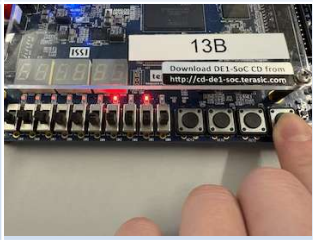
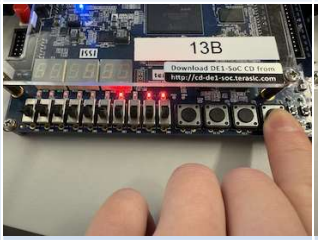
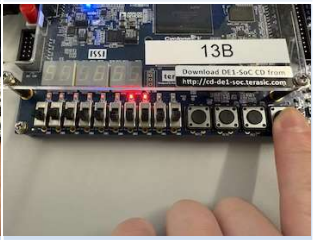
pushbutton #9	pushbutton #10	pushbutton #11	pushbutton #12
press KEY[0] 9+16k time	press KEY[0] 10+16k time	press KEY[0] 11+16k time	press KEY[0] 12+16k time
1001	1010	1011	1100
			

Table 4: Pushbutton-Controlled 4-bit Counter Result

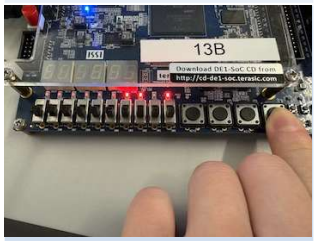
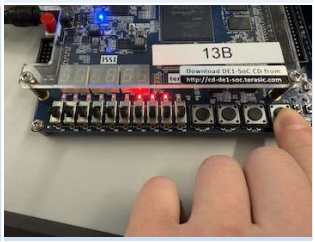
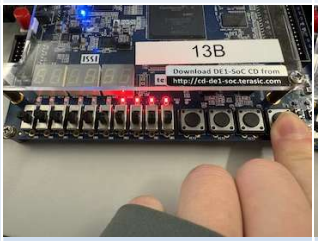
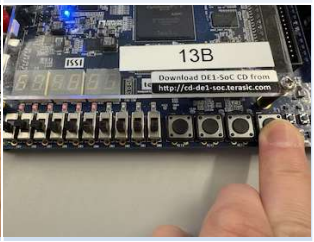
pushbutton #13	pushbutton #14	pushbutton #15	pushbutton #16
press KEY[0] 13+16k time	press KEY[0] 14+16k time	press KEY[0] 15+16k time	press KEY[0] 16+16k time
1101	1110	1111	0000
			

Table 5: Pushbutton-Controlled 4-bit Counter Result

Free-Running 4-bit Counter

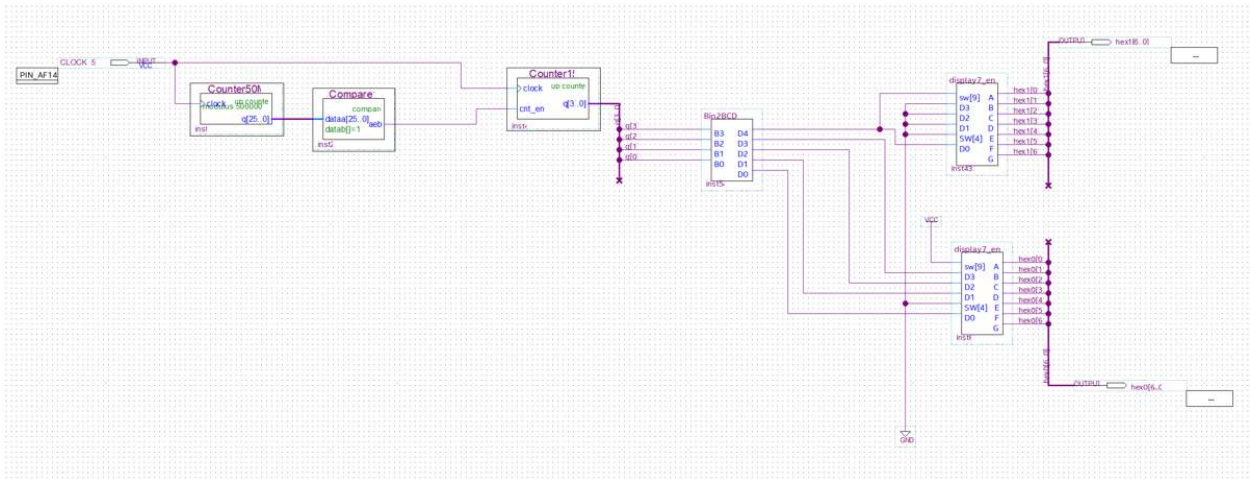


Figure 6: Free-Running 4-bit Counter (1s)

Explanation:

The 26-bit timer Counter50M is driven by the 50MHz board clock, counting from 0 up to 49,999,999(50,000,000 in total) cycles before resetting.

Compare1 acts as the pulse generator, it is constantly comparing the current value of Counter50M to constant 1. The output of Compare1 is only high for a single clock cycle out of 50,000,000 cycles, which is when Counter50M equals to 1.

Because this the High of Compare1 is connected to the cnt_en(count enable) inout of Counter15, the 4-bit counter only increments its value once every time Counter50M completes a full cycle(50,000,000 cycles). Since 50,000,000 cycles at 50MHz takes exactly 1 second, the 4-bit counter increments every per second.

Change the datab Constant

If I change the constant in Compare1 to another value (like 5), there won't be changes in frequency. Because the Compare1 still hits "5" once every 50,000,000 times. The 4-bit counter still increment once per second, it will just happen 4 clock cycles later than it did before. The overall speed remains exactly the same.

7-segment display logic Explanation

- Binary Output: Counter15 outputs a 4-bit binary number(0000 to 1111, which is 0 to 15 in decimal)

- Bin2BCD takes the 4-bit binary input and splits it into two separate decimal digits: Tens and Units. (Eg: for Binary 1010(decimal 10), it outputs a 1 for the Tens and 0 for the Units.

- The **Units** digit gets sent to the **bottom display7_en** to drive the **HEX0** display.

- The **Tens** digit gets sent to the **top display7_en** to drive the **HEX1** display.

- Control 0: since the design must not display a zero on the tens digit. This is handled by the enable pins so that if the Tens value is 0, the display remains blank.

    - **Tens display explanations**: D3, D2 and D1 are grounded because Tens only display 1 or 0, only need lowest bit D0 to display 1 or 0. sw[4] (negative signal) is grounded because we don't need negative display feature. D0 connects to D4 which is the sign bit of bin2BCD, displaying the sign of output. For the same reason, sw[9](enable signal) is connected to D4 because only when D4 is 1, Tens HEX[1] is enables, disabled when D4 bit is 0.

- **Units display explanations**: sw[9](enable signal) is high because I need to always enable the Units HEX[0]. and the other output bits D[3..0] of Bin2BCD are connected respectively to the Units display HEX[0].

Partial Increments Pictures (1s)

(see video in uploaded files)

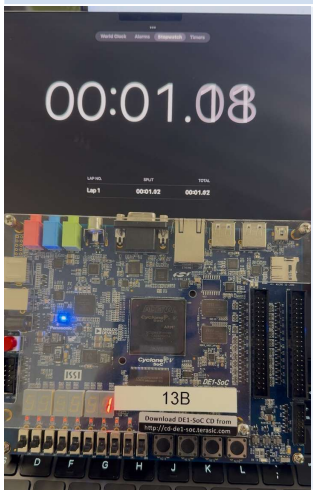
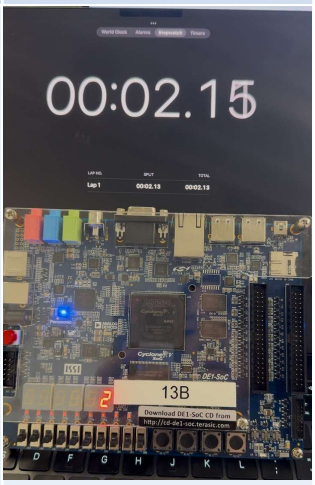
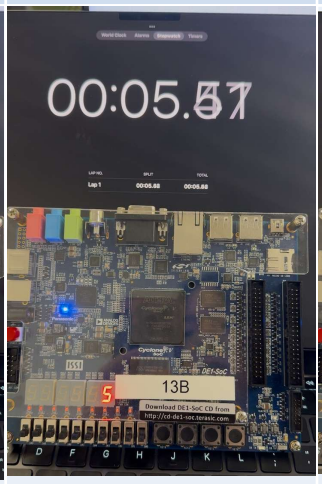
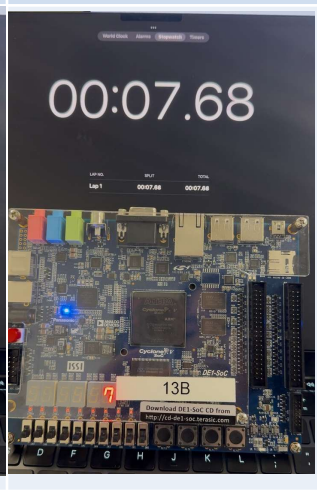
pic #1	pic #2	pic #3	pic #4
			

Table 6: Free-Running 4-bit Counter Partial Increments Pictures (1s)

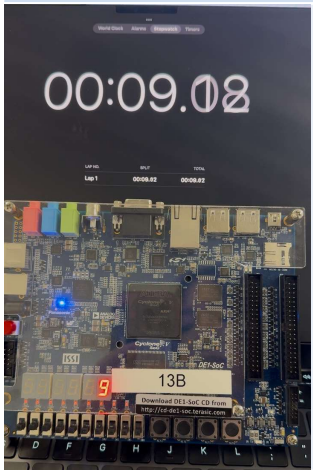
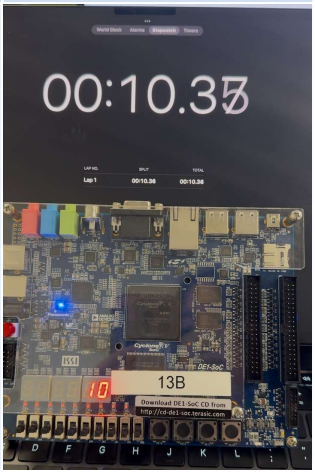
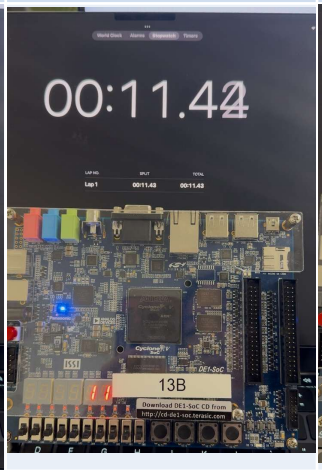
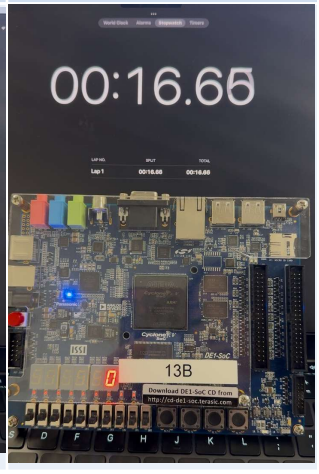
pic #5	pic #6	pic #7	pic #8
			

Table 7: Free-Running 4-bit Counter Partial Increments Pictures (1s)

Change Speed to 0.5 second per increment

(see video in uploaded files)

Modulus Value:

Change from 50,000,000 to **25,000,000**.

Number of Bits:

Keep this at **26 bits.**

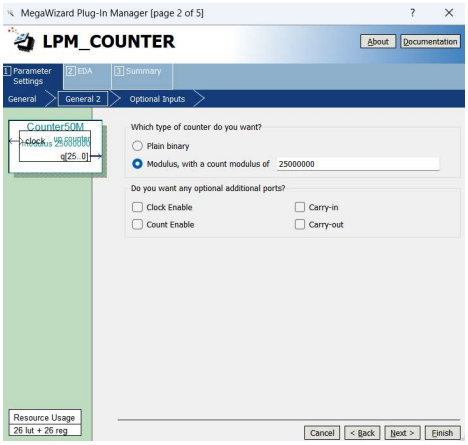


Figure 7: modification from 50,000,000 to 25,000,000

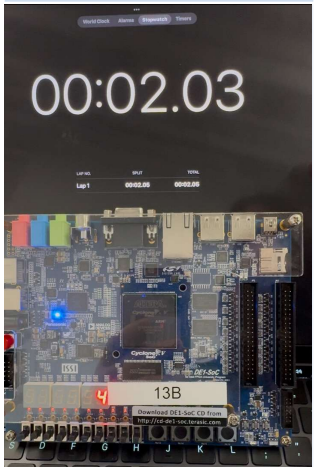
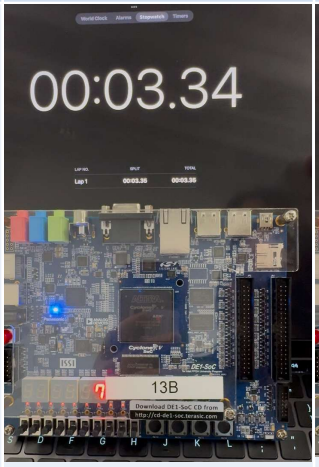
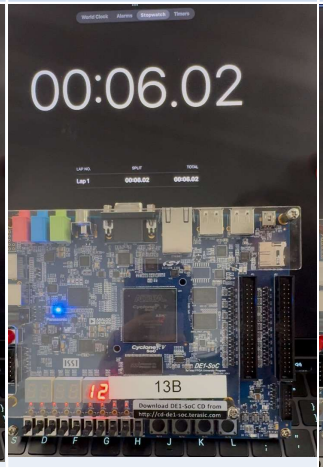
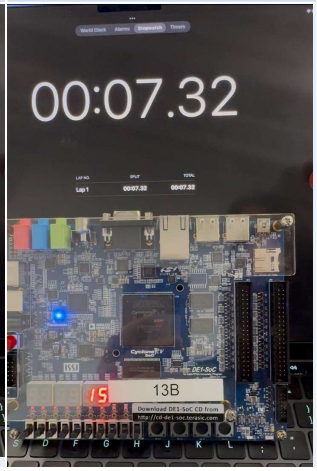
pic #1	pic #2	pic #3	pic #4
			

Table 8: Free-Running 4-bit Counter Partial Increments Pictures (0.5s)

Fully Controlled 4-bit Counter

The goal is to develop a counter that is controlled by pushbuttons and switches. You will start with the free-running 4-bit counter that you designed in the previous section. The overall functionality of the counter is controlled by the pushbutton keys and switches as follows:

SW[9]: start or stop counting
SW[8]: change the direction (i.e., increment vs. decrement)
KEY0: load counter value from switches
SW[3..0]: define the start value of the counter

Start / Stop Counter

Assignment 4

Validate the functionality of the **controlled_4bit_counter** on the DE1-SoC board. Add a screenshot of your new design in your report. Describe in your lab report how you achieved its start/stop functionality.

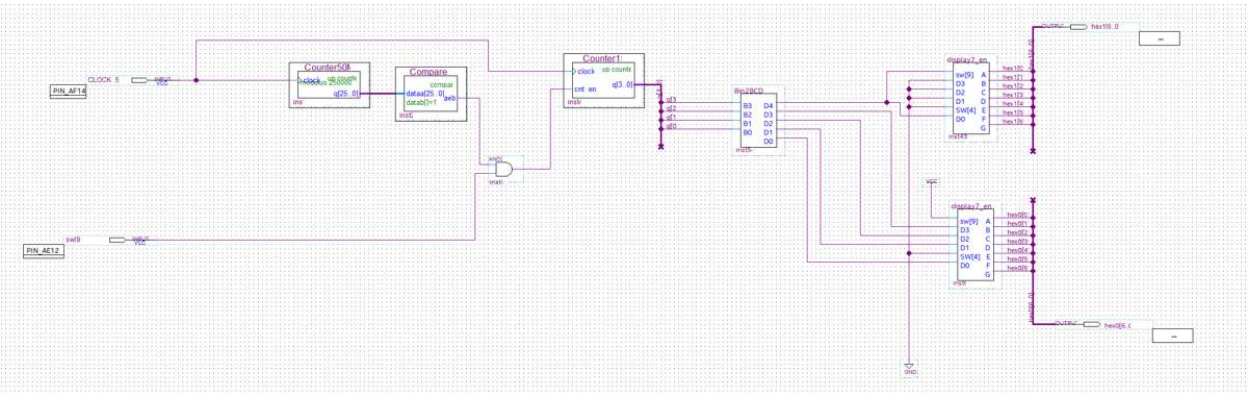


Figure 8: Start / Stop Free-Running Counter (0.5s) Schematic

Inserted an AND gate between timing logic and the main counter. One inputs of the AND gate is sw[9], the other is output of the Comapre1 block.

Only when sw[9] is ON, the AND gate allow the pulses from the Compare1 block to flow through into counter15 to increment per 0.5 second.

When sw[9] is OFF, the output of AND gate is always o regardless of the Compare1 block's output. Then counter15 enable receive a constant 0 which will stop the counter15 from proceeding.

Bi-Directional Counter

Assignment 5

Validate the up/down functionality on the DE1-SoC board. Add a screenshot of your new design in your report. Describe in your lab report how you achieved the direction control functionality.

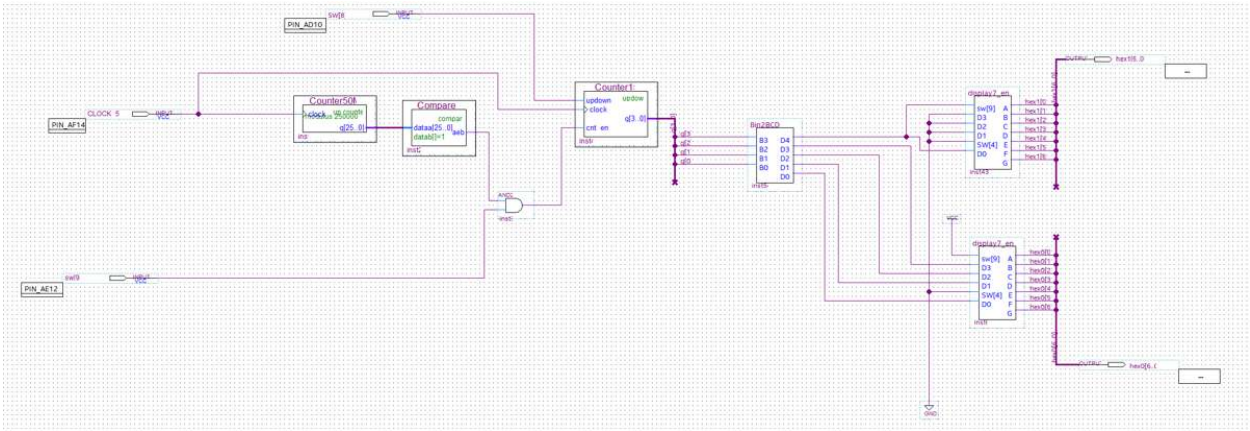


Figure 8: Start / Stop Free-Running Counter (0.5s) Schematic

Changed the Counter15 from “up only” to “up/down”.

Connected the “up/down control” sw[8] to “up/down” controller on Counter15.

Loading the Counter with a Value

Assignment 6

Validate the loading functionality on the DE1-SoC board. Add a screenshot of your new design in your report. Describe in your lab report how you achieved the load value operation functionality.

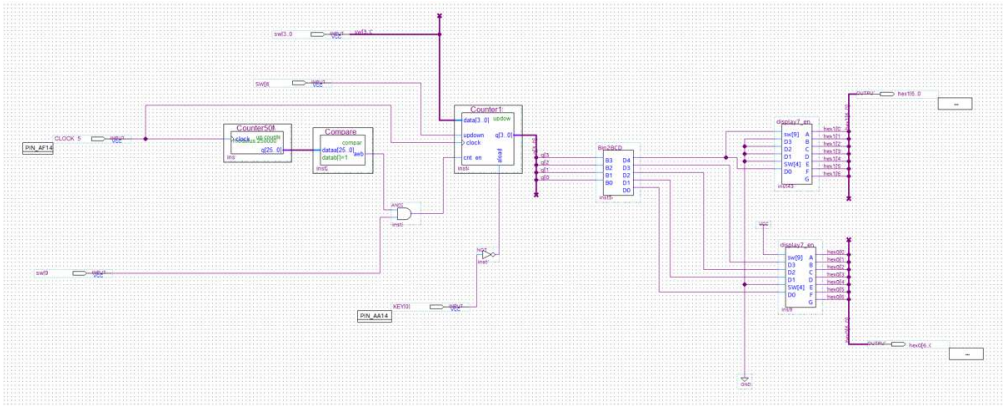


Figure 9: *Loading the Counter with a Value* (0.5s) Schematic

- use KEY[0] to load the data from sw[3..0]. KEY[0]'s complement is used to control this. when KEY[0] is not pressed, the counter counts continues; when KEY[0] is pressed, the counter connected to aload loads data from the sw[3..0].
- Counter15 now takes in 4 bit data input to initialize the starting point of the counter.
- sw[9] and sw[8] remains the same.

Results and Analysis

(Results shown above in pictures and screenshots)

Analysis

Starting with a simple 1-bit pushbutton-toggled LED, the design was built up step by step into a fully controlled 4-bit counter with multiple features.

Using a 26-bit timer counter (Counter50M) driven by the 50MHz board clock worked well for generating precise 1-second and 0.5-second timing pulses, showing that frequency scaling through a modulus counter is a reliable way to derive slower clock enables from a fast system clock.

Changing the comparison constant in Compare1 had no effect on counting frequency, only on phase, which makes sense given how the design works.

The 7-segment display logic correctly split the binary counter output into tens and units digits through the Bin2BCD component, and the enable pin logic made sure no leading zero appeared on HEX1.

The start/stop control using an AND gate, bidirectional counting through sw[8], and asynchronous loading through KEY[0] all worked as expected.

Conclusion

Each stage of the design built on the previous one, eventually resulting in a fully featured counter that could start, stop, count up or down, and load a preset value from the switches. Everything behaved as expected at each step.

References

1. Michael Benjamin, "*Lab Report Guide*", Northeastern University, January 18, 2006.
2. Given lab 5 pdf in EECE2311 canvas.